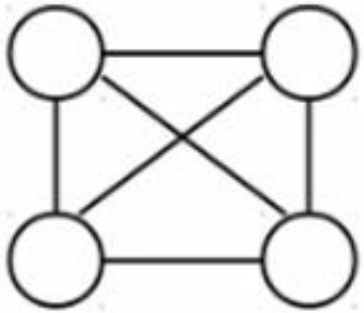
Q and A for Lecture 7

Prof. Noel Fernando

Question 1

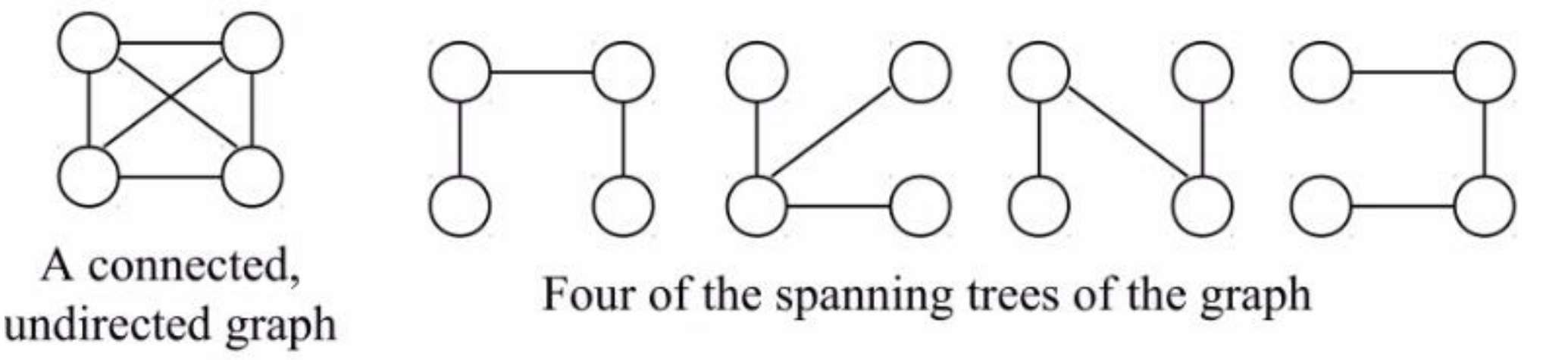
- Find all the spanning trees of the following graph.



A connected, undirected graph

Spanning Tree (cont....)

- A graph may have many spanning trees , for instance the complete graph on four vertices.



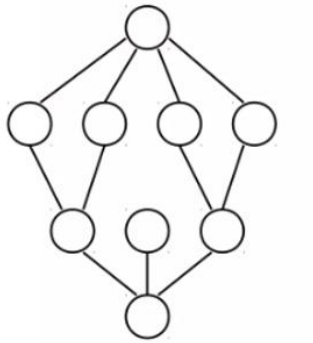
A connected, undirected graph

Four of the spanning trees of the graph

A **spanning tree** is called a **subgraph** because it is derived **from the original graph** by selecting **some of its edges** but **all of its vertices**.

Question 2

- Consider the following undirected graph.



An undirected graph

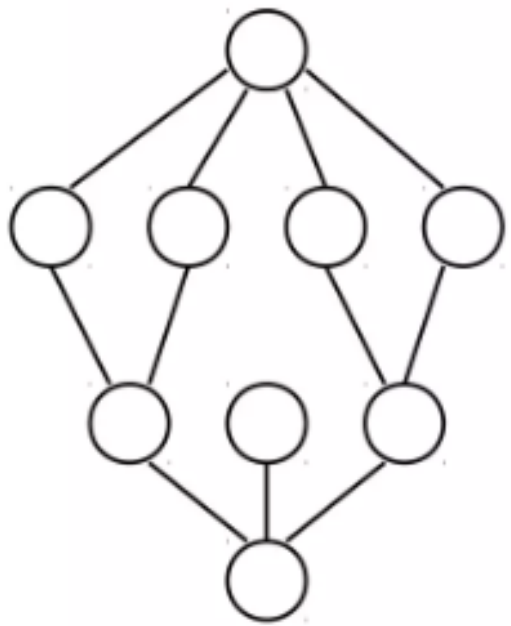
- Construct one spanning tree of the graph based on BFS and DFS traversal

To find a spanning tree of a graph,

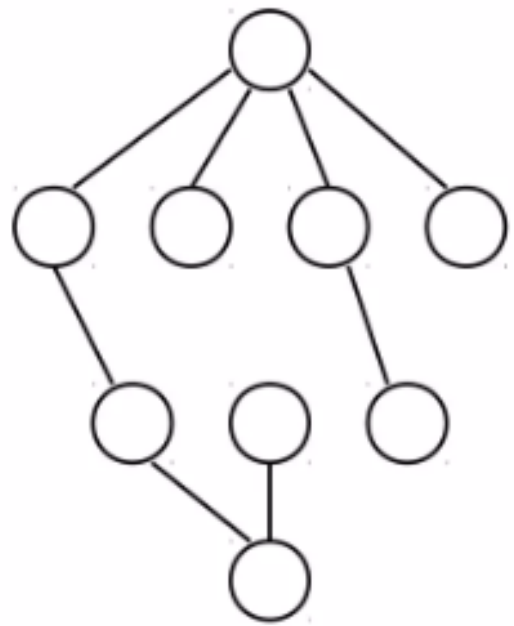
Pick an initial node and call it part of the spanning tree

do a search from the initial node:

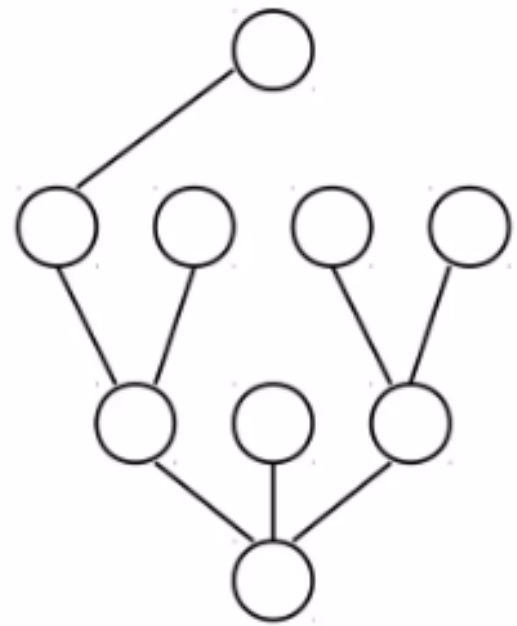
each time you find a node that is not in the spanning tree, add to the spanning tree both the new node *and* the edge you followed to get to it.



An undirected graph



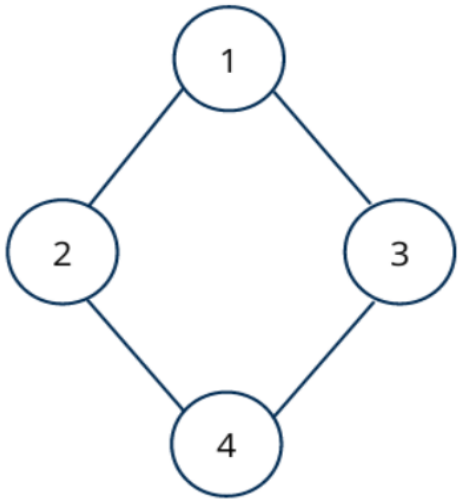
One possible result of a BFS starting from top



One possible result of a DFS starting from top

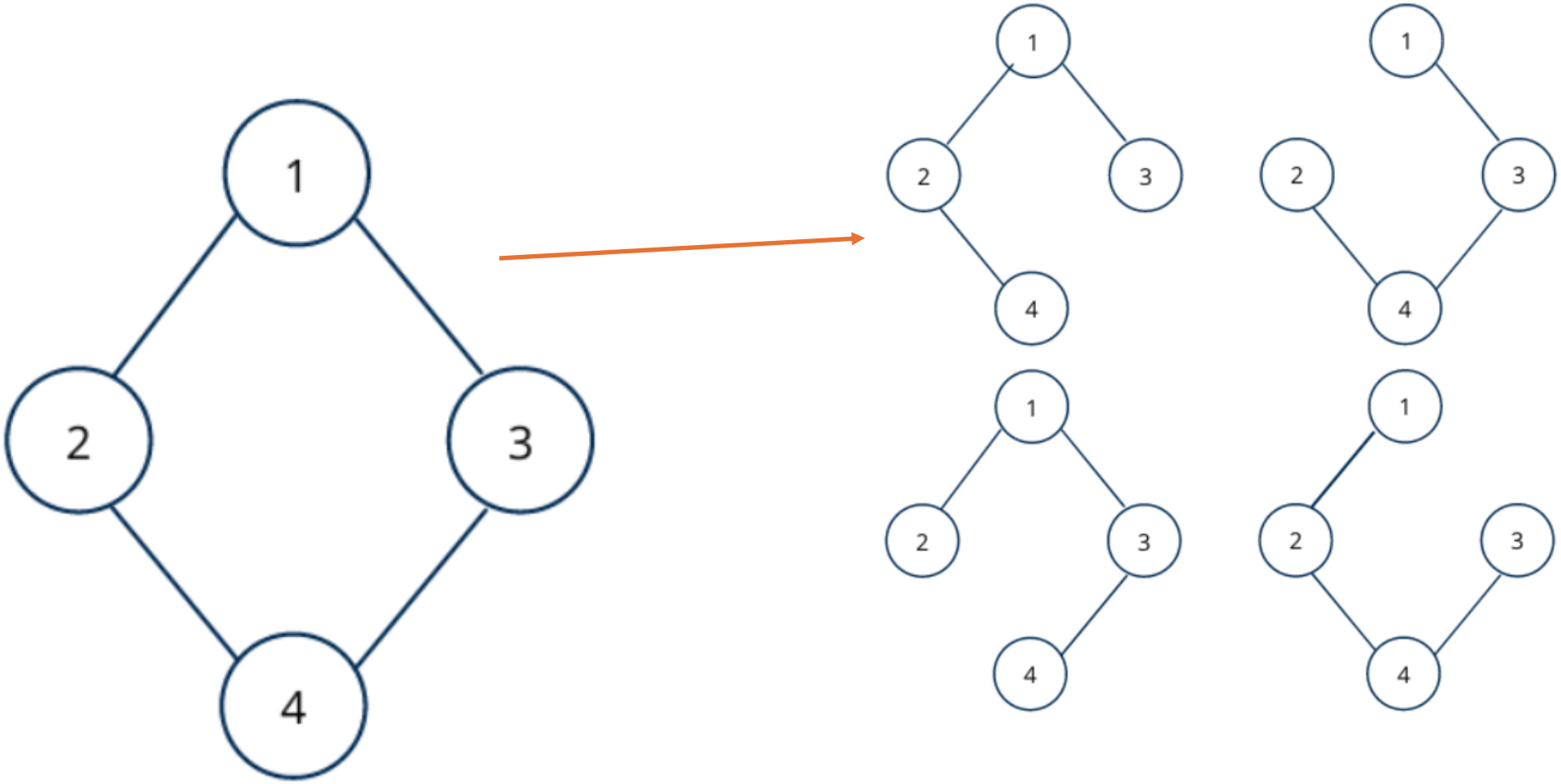
Question 3

- Consider the following undirected connected graph:



- Draw all possible spanning trees

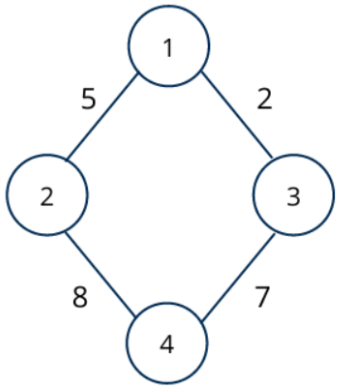
Answer to the question 3 - Possible spanning trees



Question 4

a) Define what is the minimum spanning tree?

- Consider the following graph:



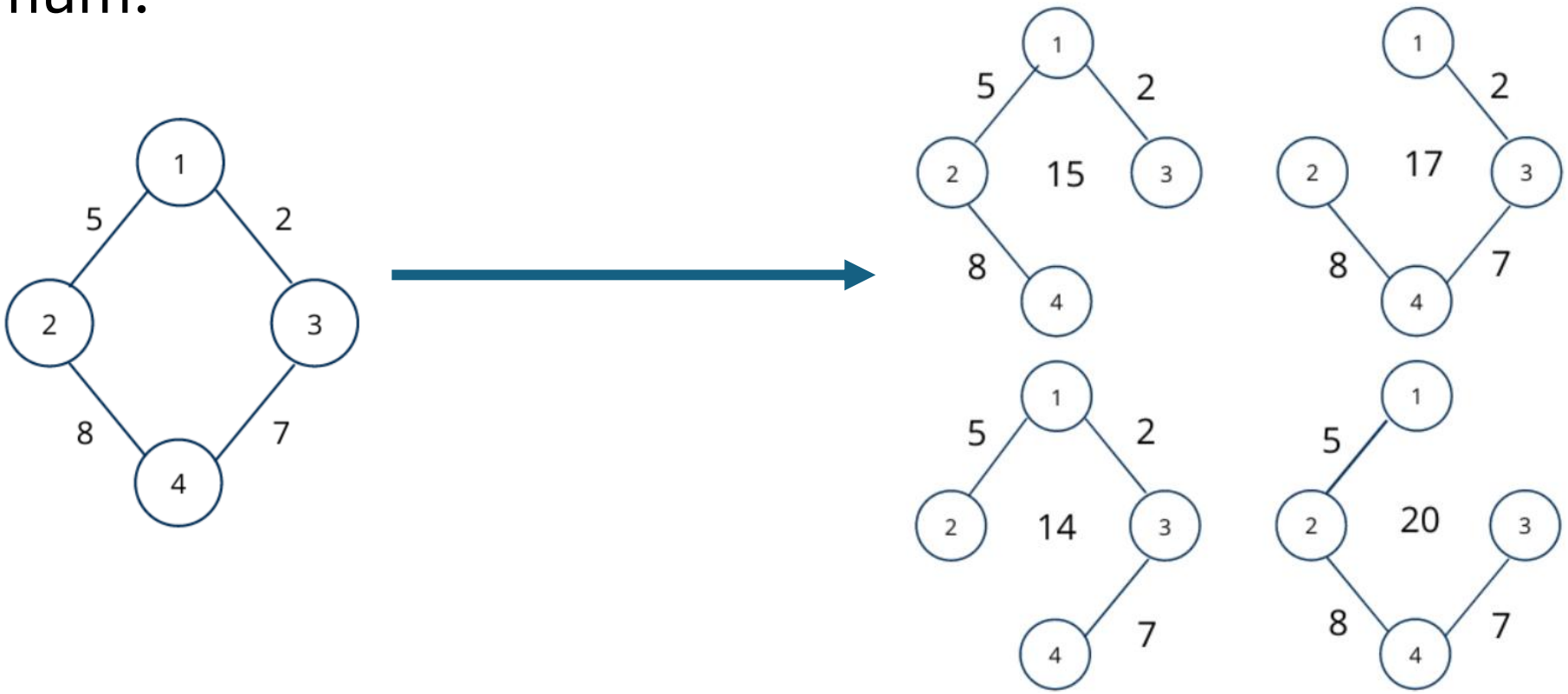
b) List all possible spanning trees

c) Find the minimum spanning tree

Answer to the question 4

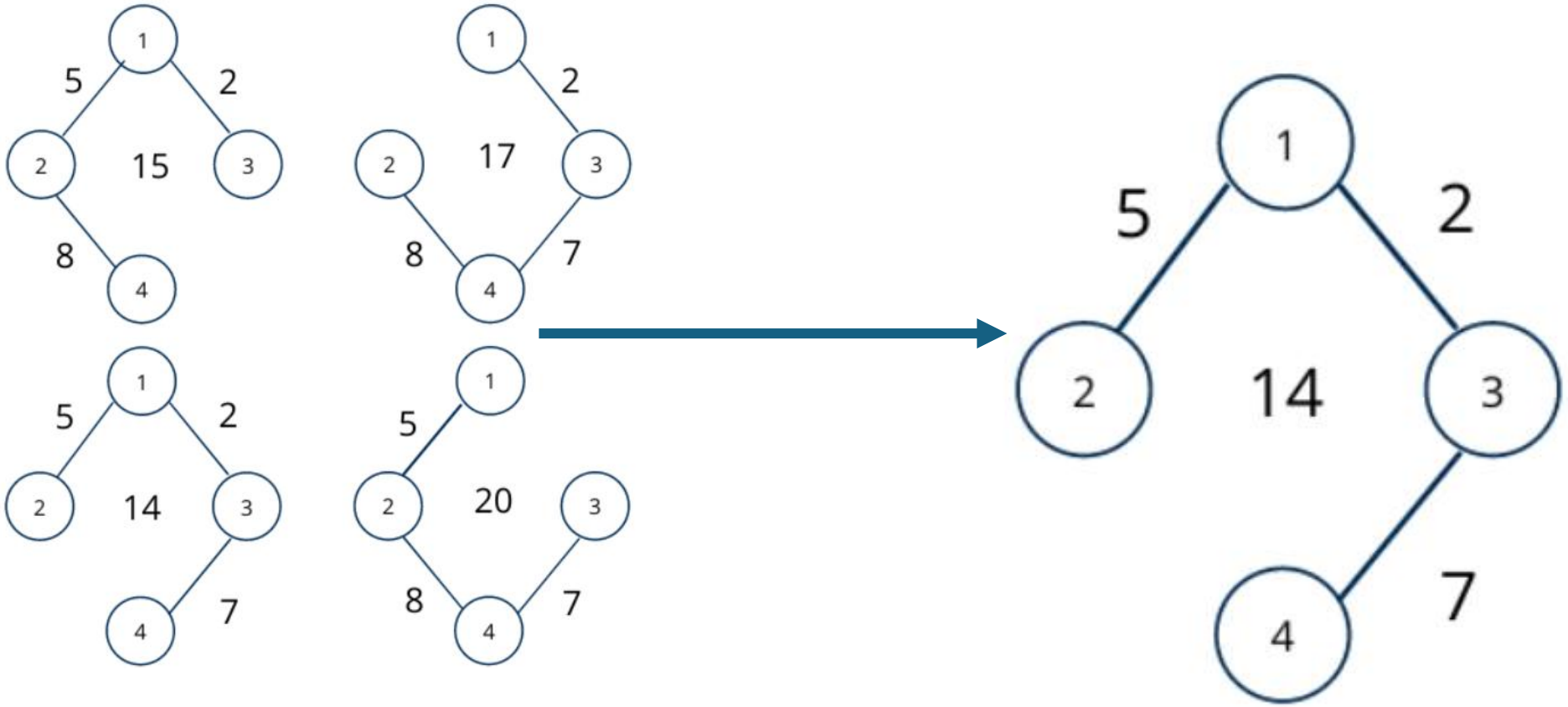
a) The minimum spanning tree of any connected weighted undirected graph is the tree whose sum of the weights of edges is minimum.

b)



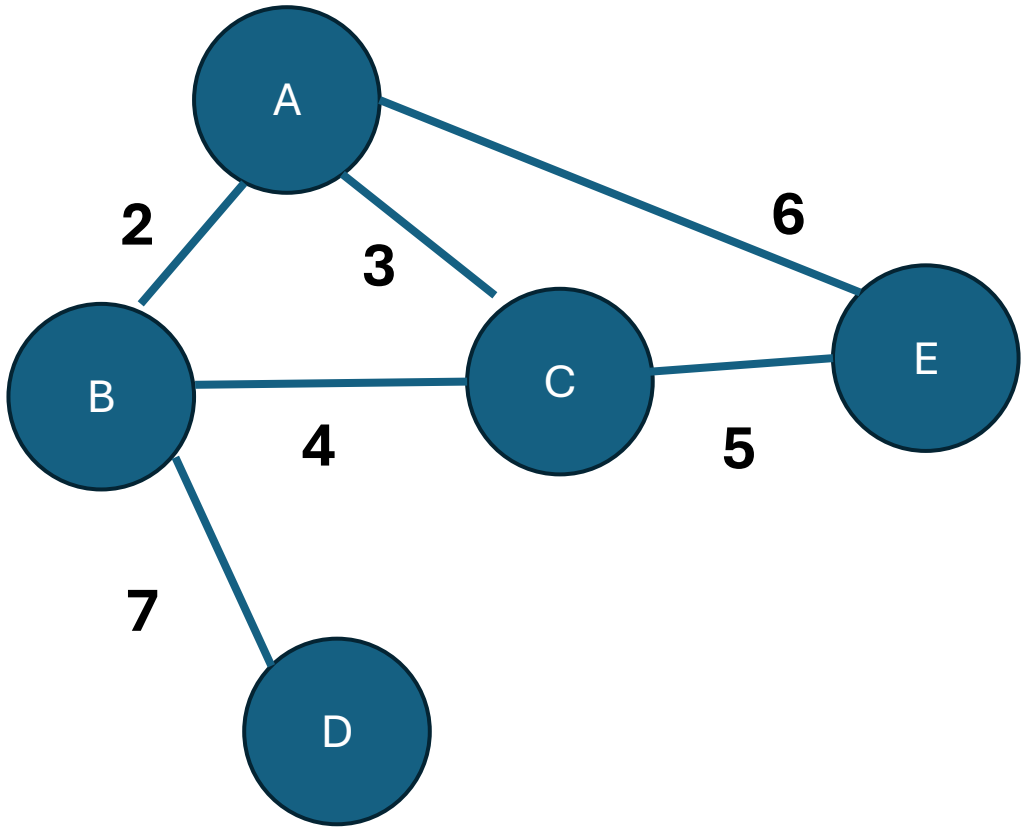
Answer to the question 4

- C)



Question 5

- How to find minimal spanning tree (MST) using Kruskal's algorithm?



Answer – Question 5

Edges with weights:

A–B = 2

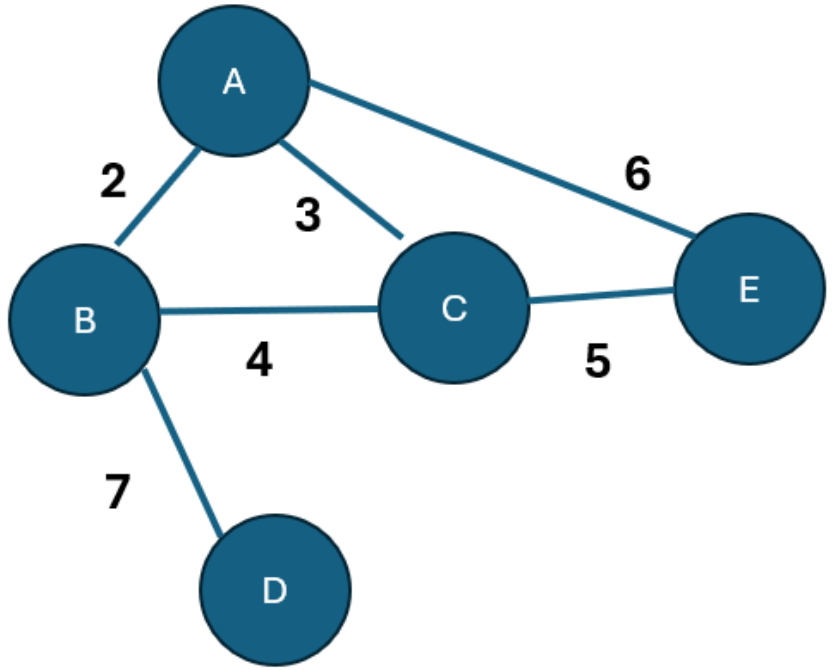
A–C = 3

A–E = 6

B–C = 4

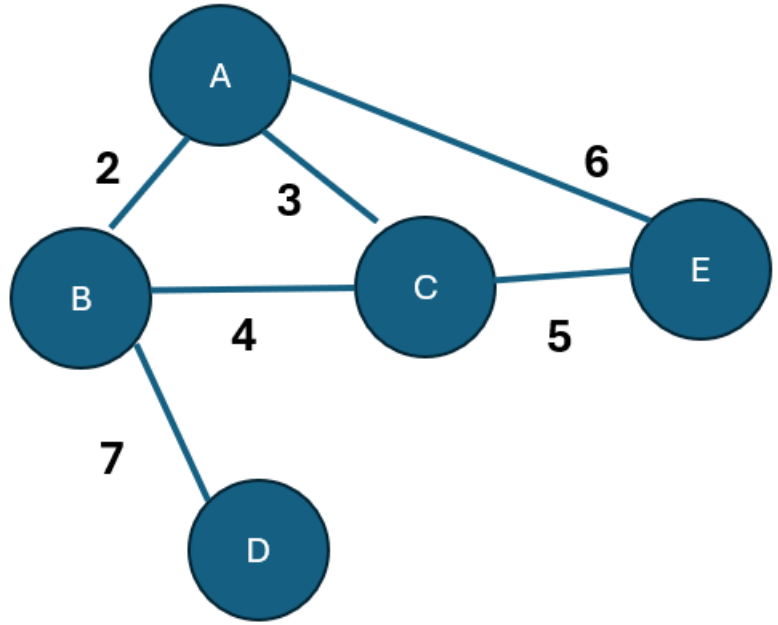
B–D = 7

C–E = 5

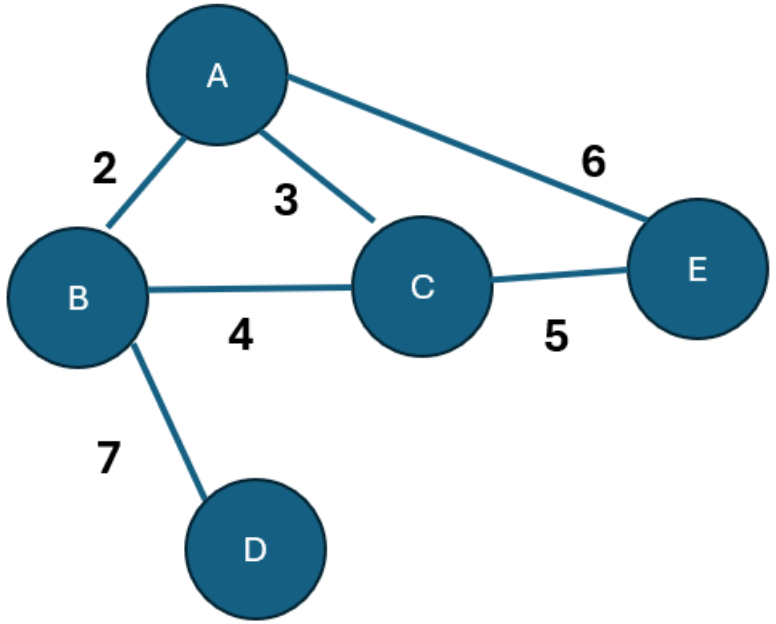


Answer – Question 5 (Cont..)

- Building the MST using **Kruskal's Algorithm**
- We have five edges. So, our edge table will have 5 columns.
- Kruskal's algorithm works by adding the smallest edge first, avoiding cycles, until we have (V – 1) edge.



Answer – Question 5 (Cont..)



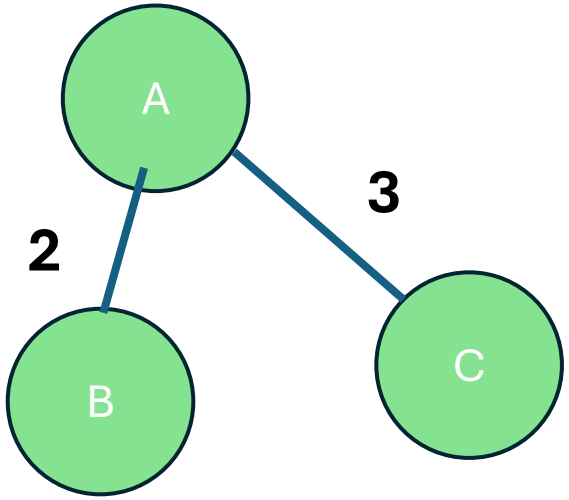
- Minimum edge is A-B (2)
-

Edge	A-B					
Weight	2					

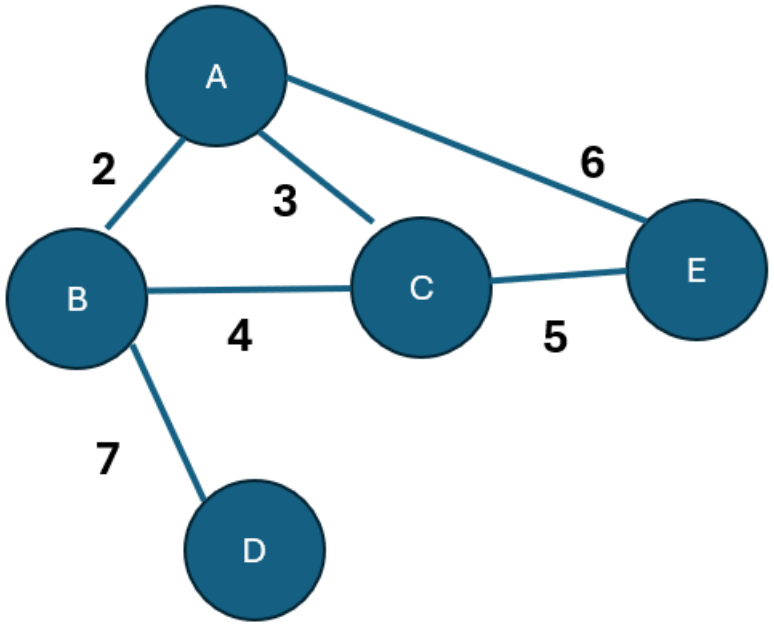
- Pick next smallest edge: A-C (3)
-

Edge	A-B	A-C				
Weight	2	3				

- **Pick next smallest edge:** B–C (4)
- X skip, it forms a cycle with A–B–C



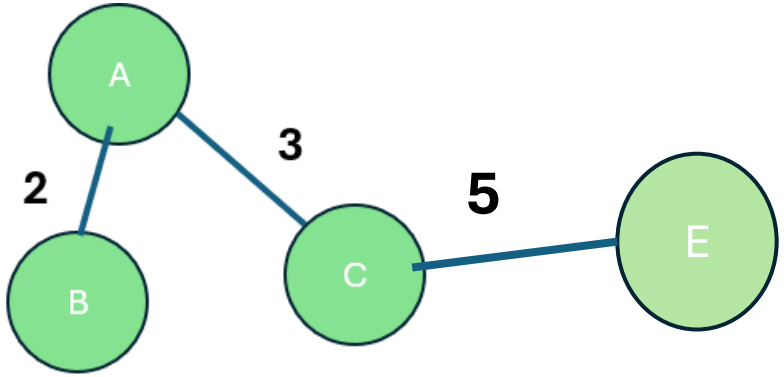
Answer – Question 5 (Cont..)



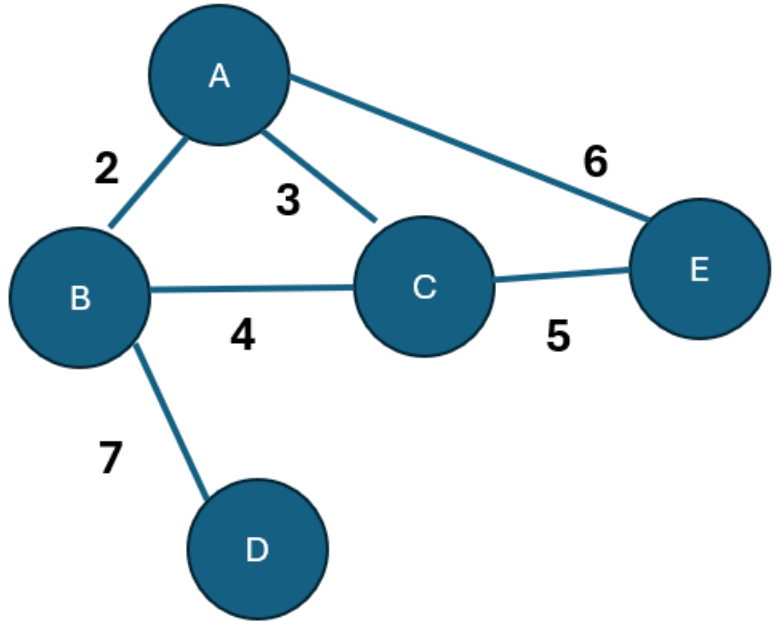
- **Pick next edge:** C–E (5)

Edge	A-B	A-C	C-E			
Weight	2	3	5			

- **Pick next edge:** A–E (6)
- X skip, it forms a cycle

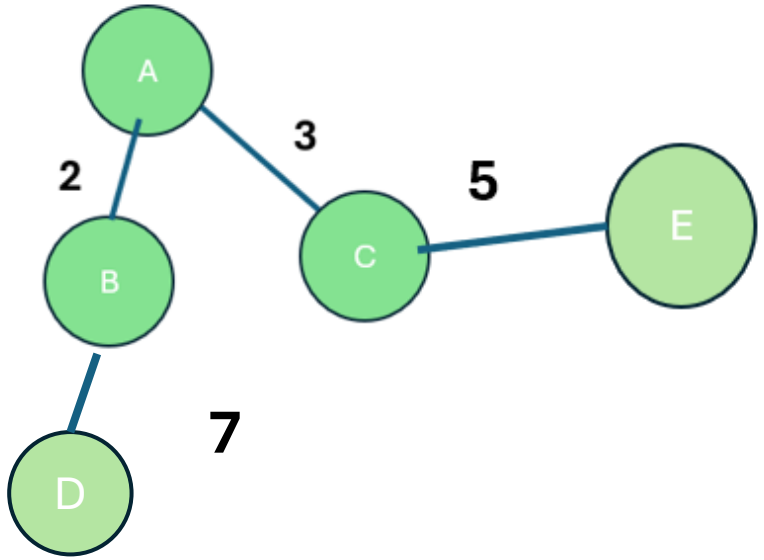


Answer – Question 5 (Cont..)



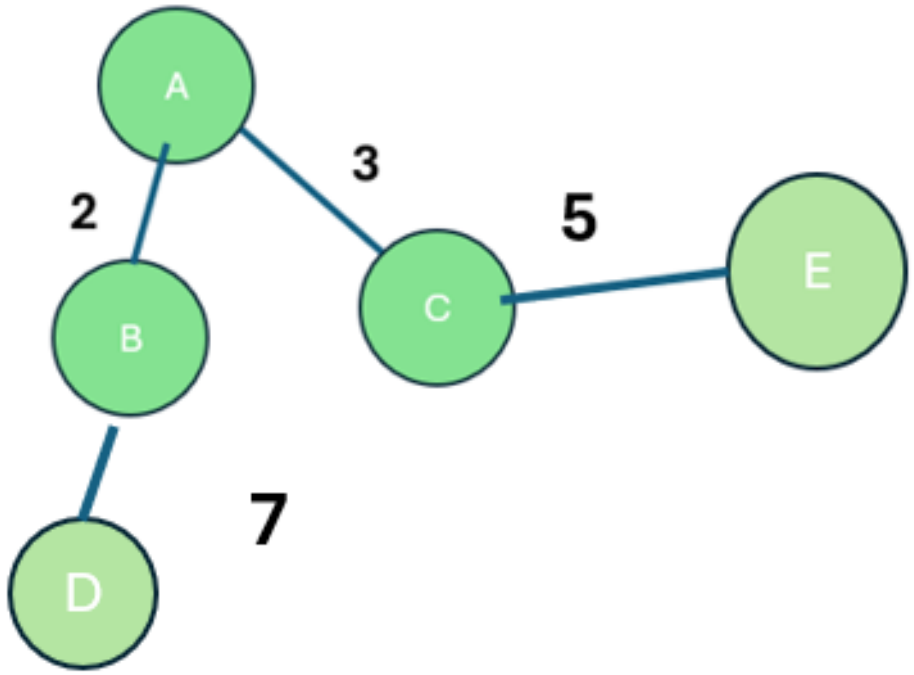
- **Pick next edge:** B–D (7)
- Final MST
- Edges in MST:
- A–B (2)
- A–C (3)
- C–E (5)
- B–D (7)

Total Minimum Weight = 2 + 3 + 5 + 7 = 17



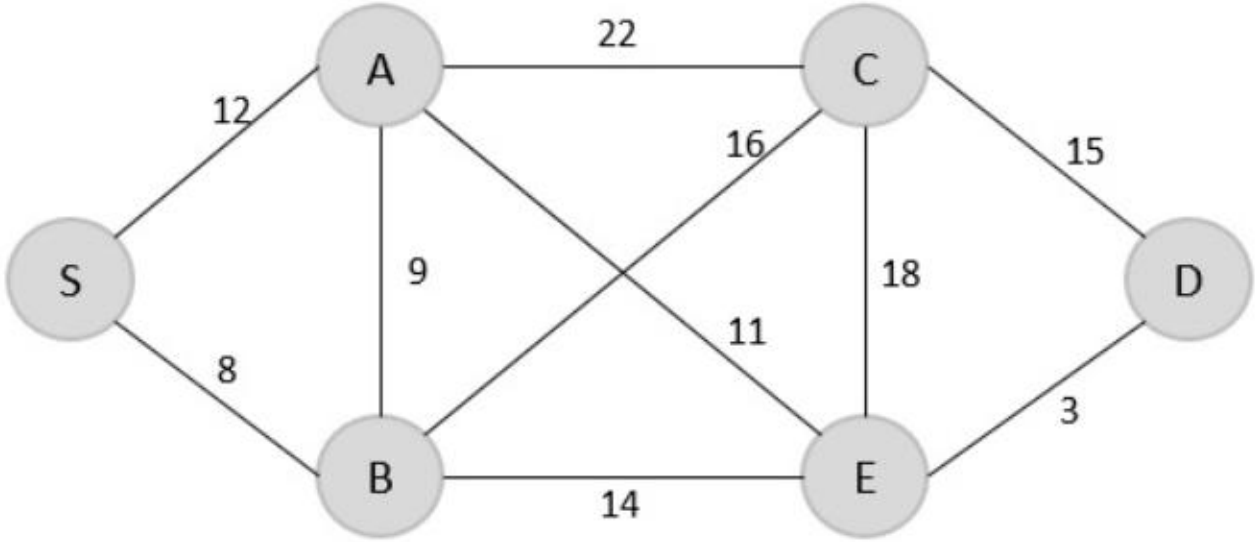
Answer – Question 5 (Cont..)

- All 5 vertices are connected
- 4 edges (V–1 = 5–1)
- Minimum weight = 17



Question 6

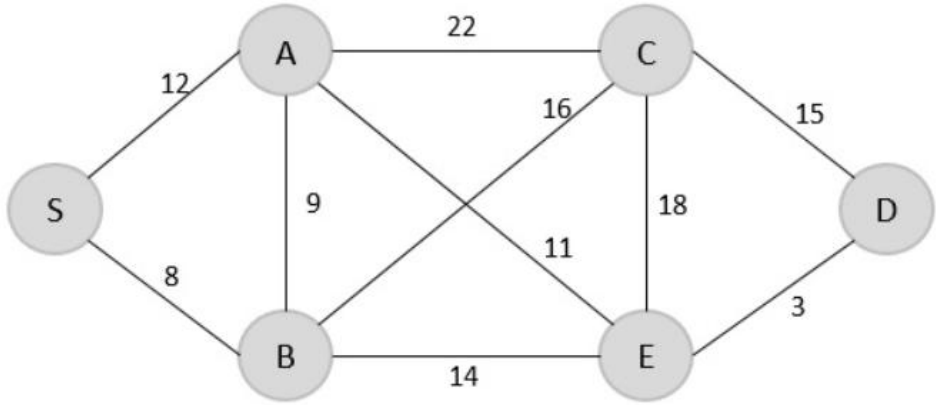
- Consider the following graph



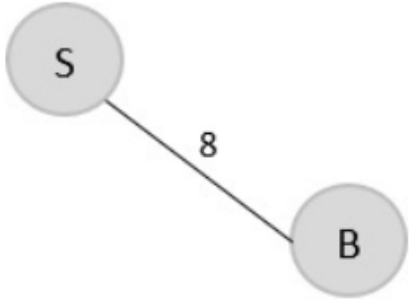
1. Find the minimum spanning tree using prims method (greedy approach) for the graph given below with S as the arbitrary root.
2. Write a C program to find the Minimum Spanning Tree (MST) using Prim's Algorithm

Prim's Algorithm for Minimum Spanning Tree (MST) Cont..

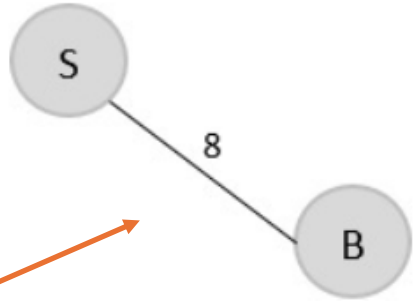
- Find the minimum spanning tree using prims method (greedy approach) for the graph given below with S as the arbitrary root.



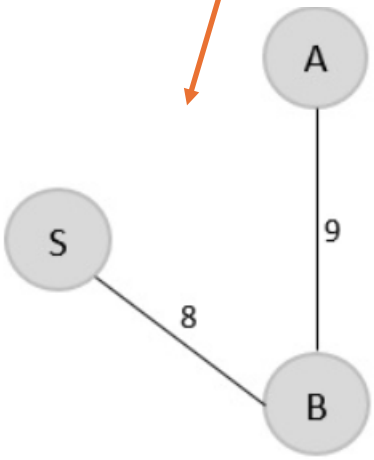
- **Step 1**
- Create a visited array to store all the visited vertices into it.
- V={ }
- Select arbitrary root (e.g., S)
- Among the all the edges, find the least cost edge
- S → B = 8 {possibilities S-A=12,S-B=8)
- V = {S, B}

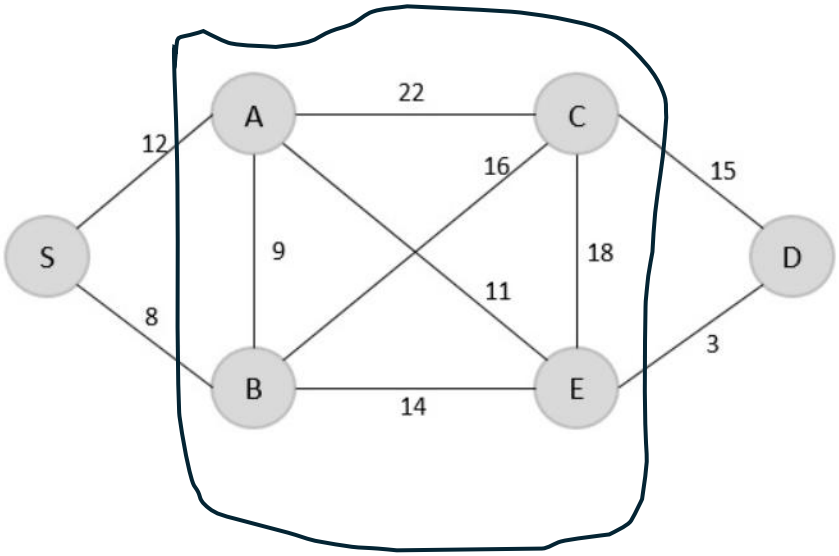


Prim's Algorithm for Minimum Spanning Tree (MST) Cont..



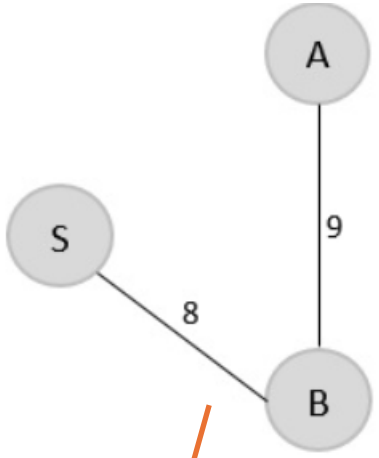
- **Step 2**
- **Last vested node is B**
- check for the least cost edge that is connected to the vertex B.
- Possibilities are : B → A = 9, B → C = 16, B → E = 14
- Minimum cost is B → A
- Update V = {S, B, A}

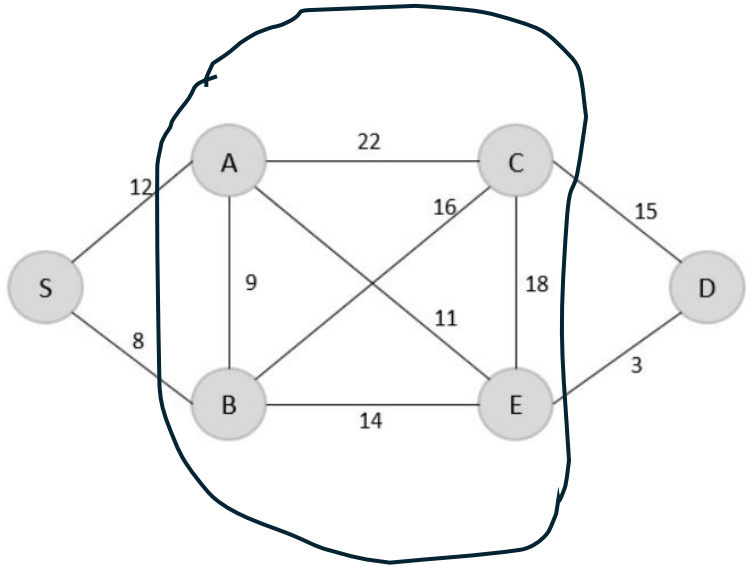


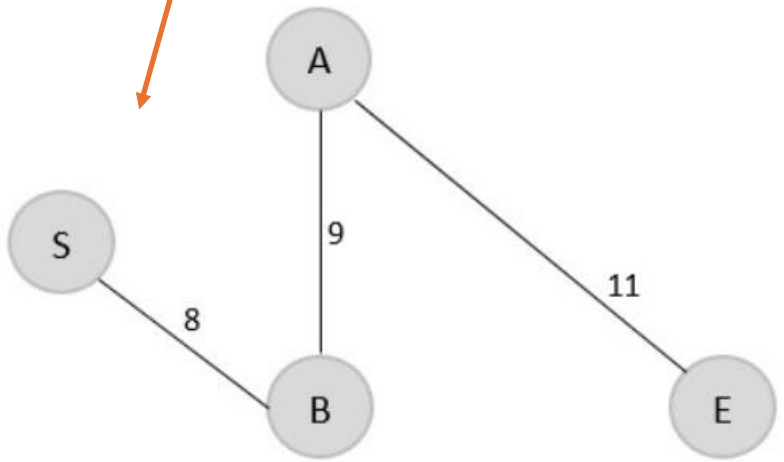


Prim's Algorithm for Minimum Spanning Tree (MST) Cont..)

- Step 3
- A is the last visited, check for the least cost edge, connected to the vertex A
- Possibilities are : A → C = 22, A → B = 9, A → E = 11
- A → B = 9 is minimum and already in MST. Next minimum is A → E = 11 and not in the MST.
- Add A → E

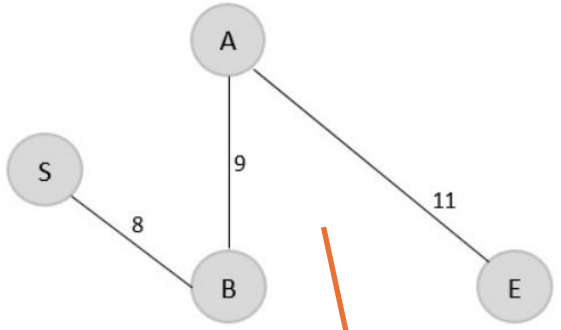


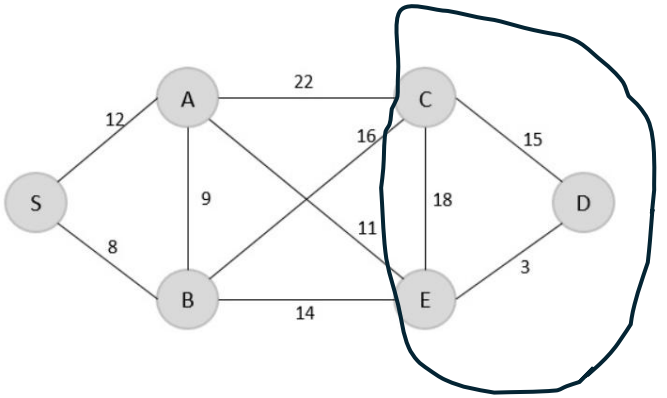


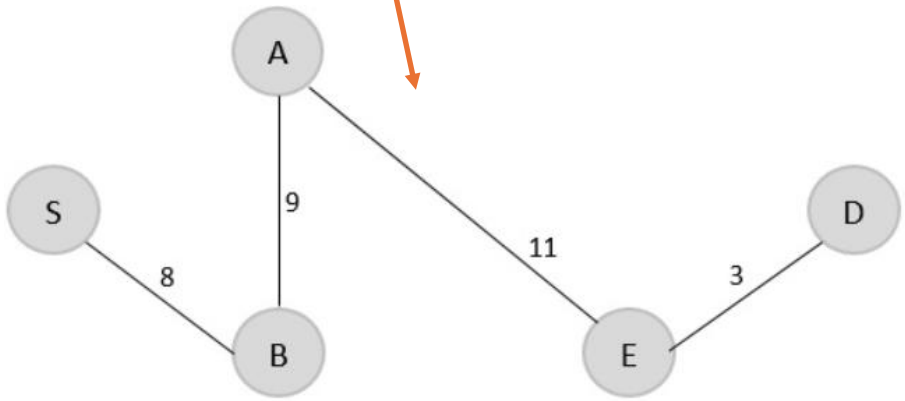


Prim's Algorithm for Minimum Spanning Tree (MST) Cont..

- **Step 4**
- E is the last visited, check for the least cost edge connected to the vertex E
- Possibilities are E → C = 18 and E → D = 3
- Least cost is E → D = 3 and add MST
- V = {S, B, A, E, D}

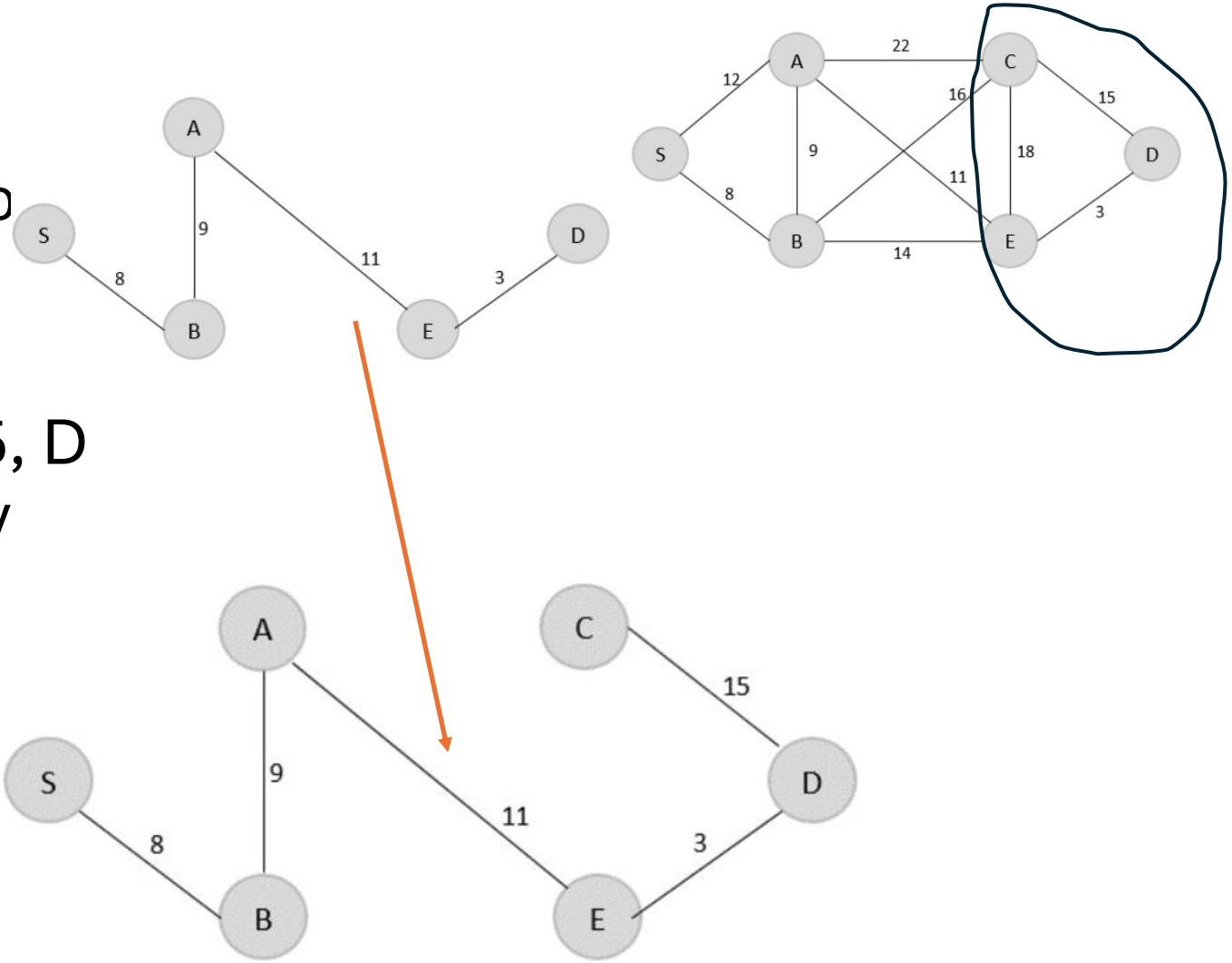






Prim's Algorithm for Minimum Spanning Tree (MST) Cont..

- **Step 5**
- D is the last visited, check fo the least cost edge that is connected to the vertex.
- Possibilities are : D → C = 15, D D→ E = 3 (D → E = 3 is already visited)
- V = {S, B, A, E, D, C}
- minimum cost = 46



C- Code - Prim's Algorithm for Minimum Spanning Tree (MST)

```
#include<stdio.h> #include<stdlib.h>
#define inf 99999
#define MAX 10
int G[MAX][MAX] = {
{0, 19, 8},
{21, 0, 13},
{15, 18, 0}
 };
```

```
int S[MAX][MAX], n;
int prims();
int main(){
    int i, j, cost;
     n = 3;
    cost=prims();
    printf("Spanning tree:");
    for(i=0; i<n; i++) {
      printf("\n");
    for(j=0; j<n; j++)
     printf("%d\t",S[i][j]);
}
```

C- Code - Prim's Algorithm for Minimum Spanning Tree (MST)

```
printf("\nMinimum cost = %d", cost);
    return 0;
}
int prims(){
int C[MAX][MAX];
int u, v, min_dist, dist[MAX], from[MAX];
      int visited[MAX],ne,i,min_cost,j; //create cost[][]
      matrix,spanning[][]
            for(i=0; i<n; i++) for(j=0; j<n; j++) {     if(G[i][j]==0)
                  C[i][j]=inf;
               else
                C[i][j]=G[i][j];
                S[i][j]=0;
      }
```

```
//initialize visited[],
distance[] and from[]
dist[0]=0;
visited[0]=1;
 for(i=1; i<n; i++) {
dist[i] = C[0][i];
from[i] = 0;
visited[i] = 0;
}
min_cost = 0;
//cost of spanning tree ne = n-1;
//no. of edges to be added
while(ne > 0) {
```

C- Code - Prim's Algorithm for Minimum Spanning Tree (MST)

```
//find the vertex at minimum distance from the tree
min_dist = inf;
for(i=1; i<n; i++)
 if(visited[i] == 0 && dist[i] < min_dist) {
v = i;
min_dist = dist[i];
}
u = from[v];
```

```
//insert the edge in spanning tree
 S[u][v] = dist[v];
S[v][u] = dist[v];
ne--;
visited[v]=1;
//updated the distance[] array
 for(i=1; i<n; i++)
if(visited[i] == 0 && C[i][v] < dist[i])
 {
dist[i] = C[i][v];
from[i] = v;
}
min_cost = min_cost + C[u][v]; } return(min_cost);
}
```